

Sistemas Operativos

2º Parcial 1C2025 – TM – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría

- 1) La TLB es una memoria caché, de mayor velocidad de acceso que la memoria principal, que almacena traducciones de direcciones lógicas a físicas con el objetivo de acelerar el proceso de traducción. De esta manera, la utilidad de la TLB radica en poder ahorrar accesos a memoria (en casos de TLB hit) que de otra manera necesitaríamos para leer tablas de páginas.
La localidad temporal es el conjunto de páginas que un proceso referencia activamente en un intervalo de tiempo. En la medida en que la cantidad de entradas de la TLB sea igual o mayor a la localidad temporal del proceso en ejecución, tendremos un gran porcentaje de TLB hits y una menor cantidad de accesos a memoria. En consecuencia, un proceso que respete una localidad temporal relativamente acotada podrá sacar un gran provecho del uso de la TLB.
- 2) Un hard link es una nueva referencia a un archivo. Por ser únicamente una nueva referencia, todos los hard links que apunten a un mismo archivo también apuntarán al mismo inodo y se diferenciarán únicamente por el nombre de la entrada de directorio.
Como contraparte, un softlink es un nuevo archivo independiente del original cuyo contenido es la ruta hacia otro archivo. Es un archivo diferente, por lo cual posee su propio inodo.

```
7777 l rwx rwx rwx 1 sisop sisop 8 2014-11-01 18:59 sl_fileloco-> fileloco
7775 - rw- r-- r-- 3 sisop sisop 0 2014-11-01 18:58 hl_fileloco_1
7775 - rw- r-- r-- 3 sisop sisop 0 2014-11-01 18:58 hl_fileloco_2
7775 - rw- r-- r-- 3 sisop sisop 0 2014-11-01 18:58 fileloco
```

Del ejemplo se puede ver que los hard links solo se diferencian por el nombre de la entrada, comparten el mismo número de inodo, son archivos de tipo regular, y toda la metadata del archivo que se ve en las entradas del directorio es la misma. Por otro lado, el softlink posee un número de inodo diferente, se ve que es un archivo de tipo "link" y la metadata del archivo que se observa en la entrada de directorio también cambia. El contador de referencias también tiene diferencias:

- Para el softlink es 1 ya que el archivo del softlink es referenciado por una única entrada de directorio
- Para los hard links es 3 ya que existen 3 entradas directorio: la "original" (que sería el primer hard link que se crea junto con la creación del archivo) y la de los otros dos hard links creados posteriormente

3)

	Sustitución local	Sustitución global
Asignación dinámica	OK	OK
Asignación fija	OK	NOK

Si la asignación de frames es fija, entonces la cantidad de frames asignados a cualquier proceso X del sistema se mantiene constante a lo largo de su “vida”. Esto se contradice con sustitución global, ya que:

- Si un proceso Y le roba un frame al proceso X, la cantidad de frames del proceso X disminuiría.
- Si el proceso X le roba un frame a otro proceso Y, la cantidad de frames del proceso X aumentaría.

4)

a) Falso. En segmentación paginada la memoria está dividida en frames, por lo que hay únicamente fragmentación interna.

b) Verdadero. En particiones fijas se divide a la memoria en N particiones de tamaño fijo, donde cada una de ellas es ocupada por un proceso diferente. En consecuencia, se limita el grado de multiprogramación a N. Esta problemática no se da en particiones dinámicas ya que el tamaño de las particiones no es fijo y la única limitante será la cantidad de memoria libre en el sistema.

5)

a) Falso. FAT no utiliza FCBs. La metadata del archivo es guardada en la entrada de directorio.

b) Falso. La utilización de una Matriz de Acceso es muy ineficiente en términos de espacio ocupado, ya que deben almacenarse los permisos para todas las combinaciones posibles de Usuario – Archivo.

Práctica

1) FS EXT2, Punteros 64 bits, Bloques 1 KiB.

a) Tam max teórico = $2^{64} * 2^{10} \text{ B} = 2^{74} \text{ B}$

Se ve limitado por la partición de 1 TiB, Tam real = 2^{40} B

ptrs x bloq = 1 KiB / 8 B = 128

Tam máx teórico archivo = $2 * (2^7)^3 * 1 \text{ KiB} = 2^{22} * 2^{10} \text{ B} = 2^{32} \text{ B} = 4 \text{ GiB}$ aprox (sin contar los punteros indirectos dobles, simples ni directos)

No se ve limitado por el tamaño del FS por lo que el real también es **4 GiB**.

b) byte 415.000 = bloque 405

El bloque 405 está direccionado por el 1er puntero indirecto doble.

→ Total de bloques leídos = 1 BD + 1 BPd + 1BPs (del doble) = **3 bloques**

c) 1 MiB / 1 KiB = $2^{20} \text{ B} / 2^{10} \text{ B} = 2^{10}$ bloques = 1024 bloques

$1024 - 10 = 1014 \rightarrow 1014/128 = 7,92... = 8$ bloques de punteros (simples)

Tengo solamente 2 punteros indirectos simples por lo que necesito acceder al doble también.

Total de bloques = 1024 BD + 2 BPs + 1BPd + 6BPs (del doble) = **1033 bloques**

2) PC 8197 (L) → P2 (PC) → DF 12293

PA 6195 (E) → P1 (PA) → DF 10291

PB 15419(L) → P3 (PB) → DF 7227

PC 4399 (L) → P1 (PC) → DF 12591

Frame	-	P2 (PC)	P1 (PA)'	P3 (PB)	P1 (PC)
0	P3(PA) UM	P3(PA) UM	P3(PA) UM	P3(PA) UM	P3(PA) M <-
1	P2 (PB) U <-	P2 (PB) U <-	P2 (PB)	P3 (PB) U	P3 (PB)
2	PO (PC) M	PO (PC) M	P1 (PA) UM	P1 (PA) UM <-	P1 (PA) M
3	P2 (PC)	P2 (PC) U	P2 (PC) U <-	P2 (PC) U	P1 (PC) U

TP Proceso A

Frame	U	M	P
2	0	1	0
2	0	1	1
2	0	0	0
0	0	1	1

TP Proceso B

Frame	U	M	P
0	0	0	0
0	0	1	0
1	0	0	0
1	0	0	1

TP Proceso C

Frame	U	M	P
2	0	1	0
3	1	0	1
3	0	0	0
0	0	1	0

- 3) a) $12538 = \text{Proceso 1, Segmento 2. Desplazamiento} = 12538 - 12288 = 250$
(2 , 250) del proceso 1
 $53152 = \text{Proceso 2, Segmento 2. Desplazamiento} = 53152 - 49152 = 4000$
(2 , 4000) del proceso 2.
 $65535 = \text{Proceso 2, Segmento 0. Desplazamiento} = 65535 - 59392 = 6143$
(0 , 6143) del proceso 2.
- b) 2048 B entra justo en la base **26.624**, cumpliendo con el criterio de Best Fit:

Base	Tamaño
0	8192
34816	8192
12288	14336
26.624	2048

En FAT no hay accesos extra a disco, por lo tanto, para leer $2048 \text{ B} / 256 \text{ B} = \mathbf{8 \text{ Bloques}}$
Para saber cuáles son los bloques del archivo, se tuvo que acceder a la FAT secuencialmente desde la entrada correspondiente al bloque 0 del archivo, hasta la correspondiente al bloque 15 del mismo, por lo tanto se realizaron **16 accesos** aunque puede considerarse que sólo 15 accesos son necesarios ya que el último bloque se conoce leyendo la entrada de la FAT del bloque anterior.